



1. Введение

Программирование сетевых приложений

Григорьев М.П.
кафедра №12 НИЯУ МИФИ
2026

О чем мы сегодня поговорим

- 1 План курса
- 2 Учебный проект
- 3 Предметная область и границы проекта
- 4 Пользователи и требования
- 5 Пользовательские сценарии и критерии приёмки
- 6 Ограничения, допущения и риски
- 7 Техническое задание и пояснительная записка
- 8 Планирование и работа в команде



О чем мы сегодня поговорим



План курса

- 1 Введение в курс
- 2 Введение в web-разработку
- 3 Введение в язык Go. Git и командная работа
- 4 HTTP. REST API на Go. Аутентификация и авторизация



План курса

- 5 Базы данных для web-разработки. PostgreSQL, Redis и S3
- 6 REST API на Go: практические задачи
- 7 Введение во frontend. Live coding
- 8 React. Разработка клиентского приложения. Live coding



План курса

- 9 Чистая архитектура. Пояснительная записка, ИПП и ПМИ
- 10 Основы Docker и CI/CD. Часть I
- 11 Основы Docker и CI/CD. Часть II
- 12 Введение в тестирование. Презентация и защита проектов



План курса

- 13 Высоконагруженные web-приложения. Часть I
- 14 Высоконагруженные web-приложения. Часть II
- 15 Безопасность web-приложений. Часть I
- 16 Безопасность web-приложений. Часть II

Аттестация: досрочная сдача проектов и приём задолженностей.



Учебный проект

В течение курса студент или команда разрабатывает web-приложение по мере изучения Go, баз данных, фронта, инфраструктуры, тестирования и безопасности.

Проект должен:

- решать прикладную задачу;
- содержать серверную часть на Go (..?);
- использовать реляционную (..) базу данных;
- иметь пользовательский web(..?)-интерфейс;
- иметь воспроизводимое окружение запуска.



Формат работы

Проект выполняется:

- индивидуально;
- командой из двух человек;
- командой из трёх человек.

До начала разработки согласуются:

- тема проекта;
- состав команды;
- зона ответственности каждого участника.

Для команды из двух или трёх человек зона ответственности каждого участника указывается в ТЗ.



Предметная область

Предметная область определяет:

- прикладную задачу;
- целевых пользователей;
- процессы, которые автоматизирует система;
- основные данные;
- ожидаемый результат работы системы.



Предметная область

Примеры предметных областей:

- бронирование помещений;
- учёт оборудования;
- электронная библиотека;
- сервис заявок;
- управление мероприятиями.



Границы проекта

В границах проекта фиксируются:

- функции, которые должны быть реализованы;
- функции, которые не входят в проект;
- внешние системы и сервисы;
- ограничения по срокам и составу команды;
- допустимый технологический стек;
- обязательные документы и способы проверки.

Все исходные материалы, документы и принятые решения должны относиться к одному проекту и соответствовать фактической реализации.



Роли пользователей

Для каждой роли определяются:

- назначение роли;
- доступные действия;
- запрещённые действия;
- доступные данные;
- операции изменения данных.

Примеры ролей:

- гость;
- зарегистрированный пользователь;
- оператор;
- администратор.

Проверка прав выполняется на стороне сервера.



Функциональные требования

Функциональное требование описывает действие системы.

Примеры:

- пользователь регистрируется в системе;
- пользователь входит по логину и паролю;
- оператор создаёт заявку;
- администратор изменяет роль пользователя;
- пользователь завершает активную сессию.

Процент готовности проекта определяется по доле реализованных и успешно проверенных функциональных требований согласованного ТЗ.



Нефункциональные требования

Нефункциональное требование описывает свойства системы и условия её работы.

Примеры:

- время ответа;
- количество одновременных пользователей;
- требования к безопасности;
- доступность;
- воспроизводимость сборки и запуска;
- поддерживаемые браузеры;
- требования к журналированию.

Нефункциональное требование должно быть измеримым или однозначно проверяемым.



Пользовательский сценарий

Пользовательский сценарий содержит:

- 1 роль пользователя;
- 2 цель;
- 3 начальные условия;
- 4 последовательность действий;
- 5 результат;
- 6 ошибочные и запрещённые варианты.

Пример: зарегистрированный пользователь входит в систему, открывает список своих сессий и завершает выбранную сессию.



Use case

Use case описывает взаимодействие пользователя с системой.

Состав:

- название;
- участник;
- предусловия;
- основной сценарий;
- альтернативные сценарии;
- постусловия;
- связанные требования.

Use case не заменяет требование, а уточняет способ его выполнения.



Критерии приёмки

Критерий приёмки определяет наблюдаемый результат проверки требования.

Пример:

- пользователь с действующей сессией может завершить её;
- после завершения сессии повторный запрос с её идентификатором отклоняется;
- операция доступна только владельцу сессии или администратору;
- результат отражается в журнале без записи секретных данных.

Для каждого обязательного сценария должны быть определены способ проверки и ожидаемый результат.



Ограничения, допущения и риски

Тип	Определение	Пример
Ограничение	Условие, которое нельзя изменить внутри проекта	Команда до трёх человек
Допущение	Предположение, принятое при проектировании	Внешний почтовый сервис доступен
Риск	Событие, которое может повлиять на срок или результат	Задержка интеграции

Для риска указываются вероятность, влияние, ответственный и способ снижения.



Методология

- Waterfall
- Agile



Waterfall

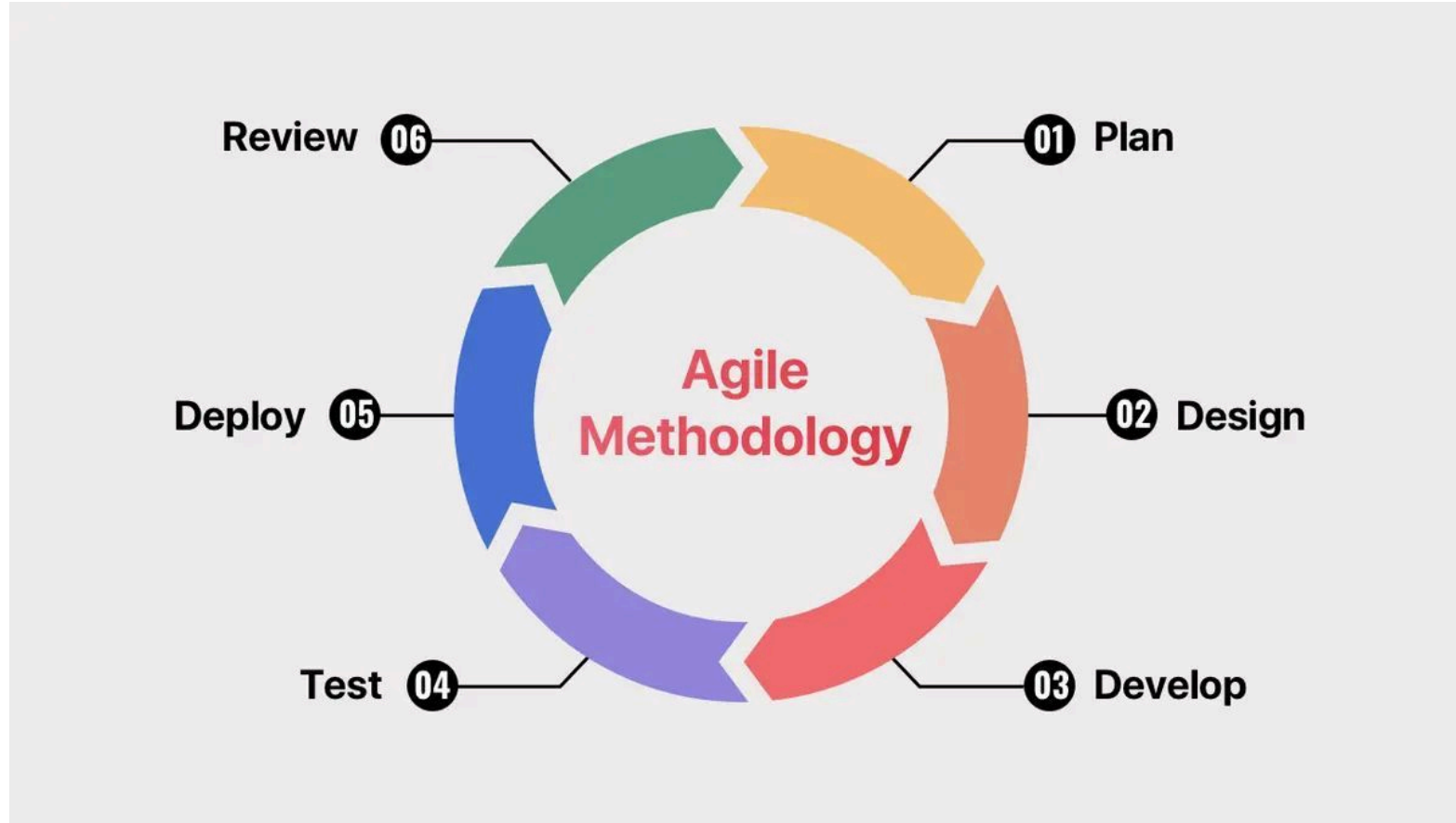
- 1 Определение требований
- 2 Проектирование системы
- 3 Реализация
- 4 Тестирование
- 5 Запуск и сопровождение



Agile

- 1 Определение требований
- 2 Проектирование системы
- 3 Реализация
- 4 Тестирование
- 5 Запуск и сопровождение
- 6 Повторение цикла с учётом обратной связи

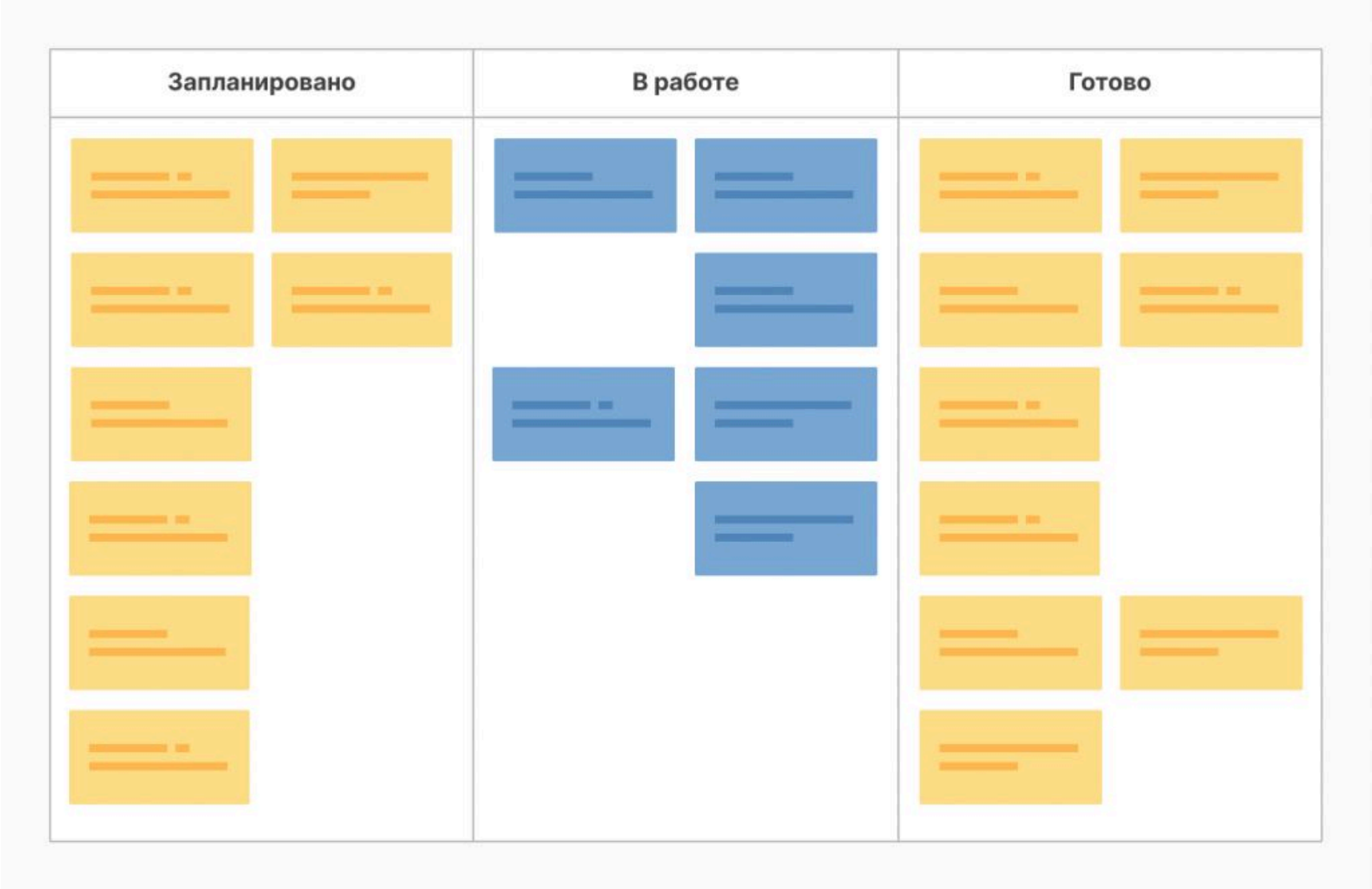




Agile | Kanban

- 1 Задачи формулируются в виде карточек на доске;
- 2 Каждая карточка содержит требование, ответственного, срок и критерий завершённости;
- 3 Карточки перемещаются по колонкам: “Запланировано”, “В работе”, “На проверке”, “Готово”;
- 4 Команда регулярно проводит встречи для обсуждения прогресса, проблем и планирования следующих шагов;
- 5 Приоритеты задач могут меняться в зависимости от обратной связи и изменений требований;



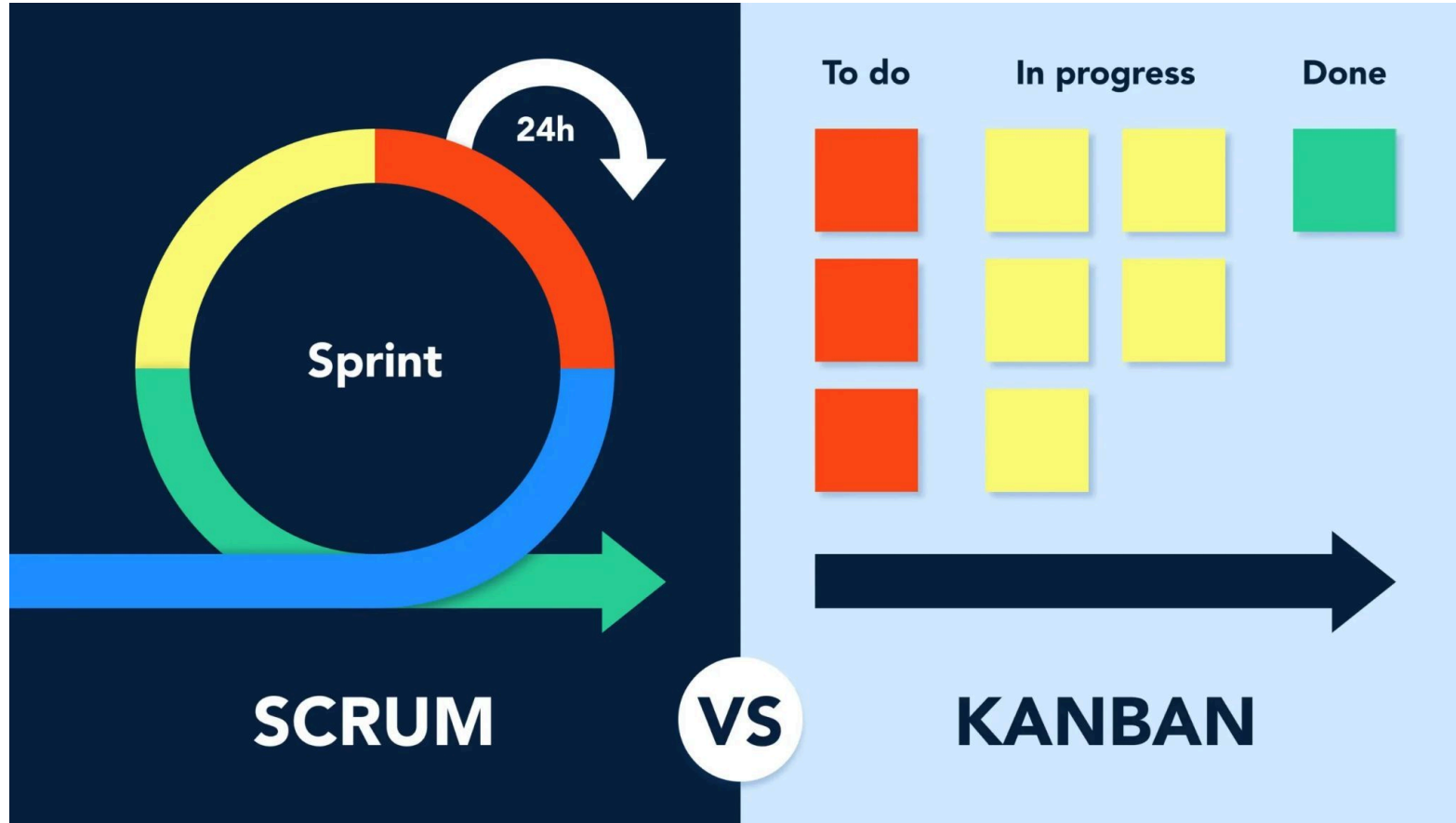


Agile | Scrum

- 1 Проект делится на спринты (обычно 2-4 недели);
- 2 В начале каждого спринта команда планирует задачи, которые будут выполнены;
- 3 В течение спринта команда работает над задачами, проводит ежедневные встречи для обсуждения прогресса и проблем;
- 4 В конце спринта проводится демонстрация выполненных задач и ретроспектива для анализа процесса и улучшения работы в следующем спринте;
- 5 Приоритеты задач могут меняться в зависимости от обратной связи и изменений требований



Agile | Kanban vs Scrum



Техническое задание

ТЗ должно быть подготовлено и согласовано в течение первых трёх недель семестра.

- введение;
- наименование проекта;
- характеристику области применения;
- назначение системы;
- цель разработки;
- целевых пользователей и их роли;
- функциональные и нефункциональные требования;
- состав серверной и клиентской частей;
- требования к безопасности;
- ограничения проекта;
- распределение задач между участниками;
- два измеримых технико-экономических показателя.



Зона ответственности участника

Для каждого участника определяются:

- реализуемые функции;
- связанные серверные компоненты или методы API.

Формулировка должна позволять определить личный вклад по ТЗ, задачам, pull request и коду.



Изменение технического задания

После согласования с преподавателем допускается изменение:

- предметной области;
- существенных требований;
- технологического стека.

Изменение фиксируется:

- в новой версии ТЗ;
- в отдельном перечне изменений.

ТЗ, пояснительная записка, ИПП, ПМИ и реализация должны соответствовать друг другу.



Обязательные требования к проекту

- регистрация пользователей;
- аутентификация и авторизация с использованием JWT;
- ролевое разграничение доступа;
- проверка прав на стороне сервера;
- защита аутентификации от перебора паролей;
- хеширование паролей с индивидуальной солью и секретным перцем;
- запрет передачи и хранения паролей в открытом виде.



Обязательные требования к проекту

- обработка штатных, ошибочных, граничных и запрещённых сценариев;
- создание схемы базы данных миграциями;
- воспроизводимая загрузка демонстрационных данных;
- доступность обязательных сценариев через UI;
- доступность обязательных сценариев через документированный API;
- сборка и запуск из чистой копии репозитория по инструкции без исправления исходного кода;
- развёртывание на выделенном сервере, если исключение не согласовано.



Дополнительное направление безопасности

Команда выбирает не менее одного направления:

- шифрование хранимых данных;
- HTTPS и управление сертификатами, включая собственную инфраструктуру сертификации;
- просмотр, завершение и отзыв пользовательских сессий;
- модель угроз разрабатываемой системы.

Направление фиксируется в ТЗ и отражается в реализации или проектной документации.



Git-репозиторий

Проект ведётся в одном Git-репозитории с понятной структурой.

В репозитории должны быть:

- README с назначением проекта, требованиями к окружению и командами запуска, сборки и тестирования;
- задачи или issue для крупных этапов и дефектов;
- отдельные ветки и содержательные pull request для существенных изменений;
- небольшие осмысленные коммиты;
- защищённая основная ветка или обязательная проверка изменений другим участником;
- зафиксированные версии основных зависимостей.



Git-репозиторий

В репозитории должны быть:

- исходный код серверной и клиентской частей;
- миграции;
- автоматизированные тесты и инструкция по их запуску;
- Dockerfile;
- конфигурация Docker Compose;
- файлы CI/CD.

В репозитории запрещено хранить:

- секреты, рабочие пароли и токены;
- приватные ключи;
- сгенерированные артефакты.



CI и тестирование

CI должен выполнять:

- статический анализ;
- тестирование;
- сборку.

Критические сценарии покрываются тестами.

Известные критические дефекты устраняются до итоговой защиты.



Комплект к итоговой защите

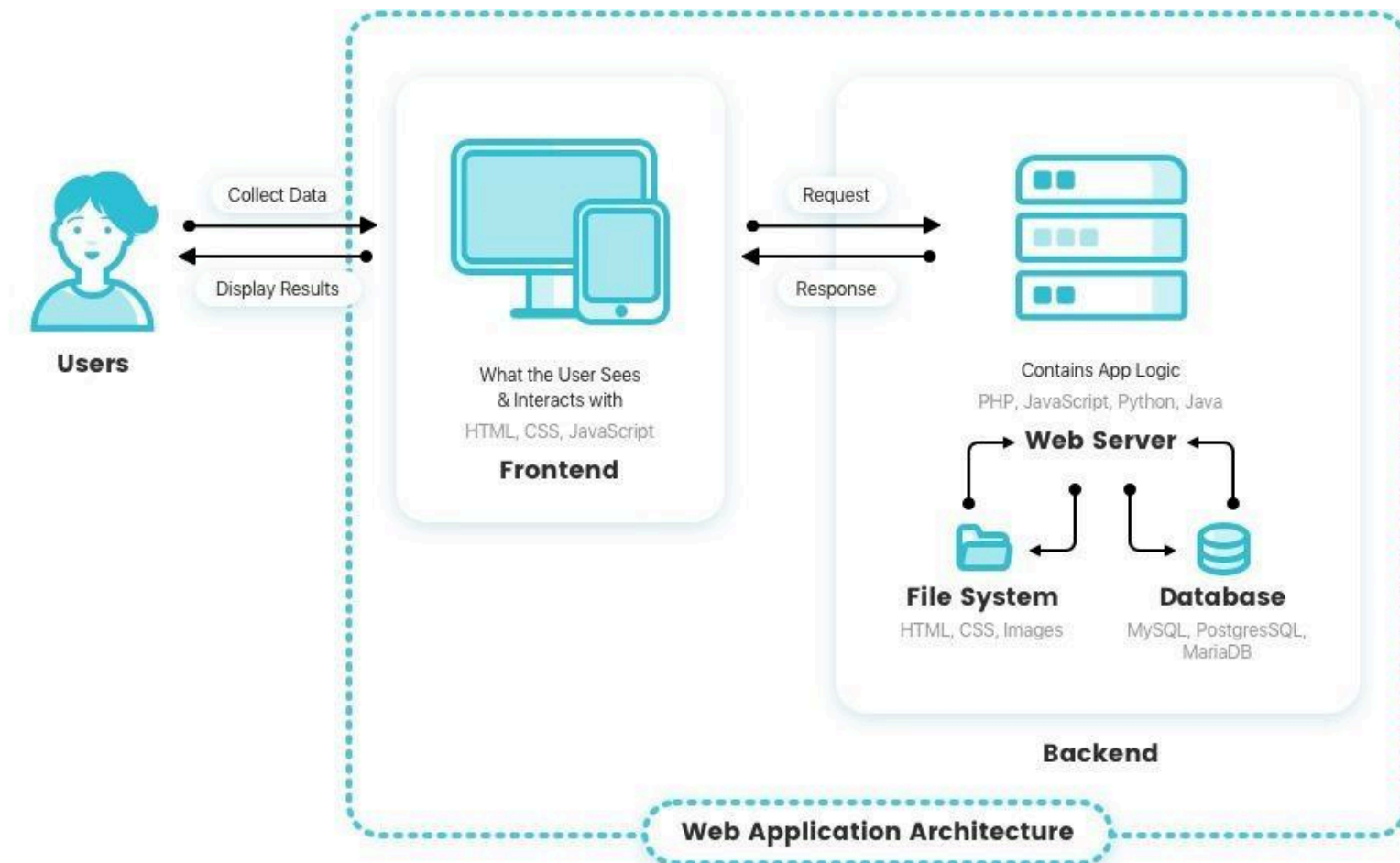
- исходный код и Git-репозиторий;
- техническое задание;
- пояснительная записка;
- программа и методика испытаний (ПМИ);
- инструкция пользователя программы (РП/РА или ИПП);
- демонстрационный набор данных без персональной и конфиденциальной информации;
- финальная презентация;
- воспроизводимое окружение запуска;
- результаты испытаний.



Этапы проекта

Этап	Результат	Состав работ
1	Базовый сценарий	Архитектура, основные модели данных, регистрация, аутентификация, начальный сквозной пользовательский сценарий
2	Основная функциональность	Бизнес-логика, взаимодействие серверной и клиентской частей, обработка ошибок, проверка прав
3	Итоговая версия	Оставшаяся функциональность, исправление дефектов, документация, итоговые испытания, ИПП и ПМИ

Так а от нас то что требуется



Итоговая защита

Каждый студент:

- демонстрирует свою часть проекта;
- показывает соответствующий код;
- объясняет технические решения;
- отвечает на вопросы по архитектуре, API, данным и безопасности;
- по требованию преподавателя вносит небольшое изменение.

Каждый участник должен понимать общую архитектуру и взаимодействие компонентов, а не только собственный код.



Допуск к защите

К итоговой защите допускается проект, который:

- соответствует согласованному ТЗ;
- соответствует обязательным требованиям регламента;
- воспроизводимо запускается;
- не содержит критических дефектов;
- сопровождается актуальными документами.



Академическая добросовестность

Запрещаются:

- копирование чужого проекта или существенных фрагментов без указания источника и понимания решения;
- передача своей работы для копирования;
- подмена личного вклада чужими коммитами;
- использование языковых моделей и кодогенерации в оцениваемой части проекта без отдельного разрешения.



Использование ИИ

Если преподаватель не установил иное, ИИ допускается для:

- генерации тестовых данных;
- работы с формулировками;
- разбора документации.

Использование ИИ необходимо раскрыть.

Студент обязан объяснить каждое принятое решение.



Оценивание

Оценивание учитывает:

- полноту выполнения ТЗ;
- работоспособность и воспроизводимость проекта;
- качество реализации и документации;
- соблюдение требований безопасности;
- результаты контрольных мероприятий;
- подтверждённый личный вклад (коммиты наши родные).

Неспособность объяснить собственный код, архитектуру, запросы к базе данных, инфраструктуру или меры защиты означает невыполнение соответствующего пункта. Защита может быть остановлена, а проект направлен на повторную сдачу, вплоть до смены темы.



Роли в команде

В команде могут быть распределены функции:

- анализ требований и ведение ТЗ;
- backend и API;
- frontend;
- проектирование и миграции базы данных;
- инфраструктура и CI/CD;
- тестирование;
- безопасность;
- документация.

Один участник может (придётся) выполнять несколько функций.
Личная зона ответственности фиксируется в ТЗ.



Структура технического задания

- 1 Назначение системы
- 2 Пользователи и роли
- 3 Функциональные требования
- 4 Нефункциональные требования
- 5 Архитектура, данные и API
- 6 Требования к безопасности
- 7 Ограничения и допущения
- 8 Этапы и распределение работ
- 9 Технико-экономические показатели



Структура пояснительной записки

- 1 Титульный лист
- 2 Реферат или аннотация
- 3 Содержание
- 4 Введение
- 5 Анализ предметной области и требований
- 6 Проектирование архитектуры, данных и API
- 7 Реализация
- 8 Безопасность
- 9 Развёртывание и эксплуатация
- 10 Тестирование и результаты
- 11 Заключение
- 12 Список использованных источников
- 13 Приложения



Нормативные документы

Обозначение	Назначение
ГОСТ 7.32-2017	Отчёт о научно-исследовательской работе. Структура и правила оформления
ГОСТ 19.201-78	Техническое задание. Требования к содержанию и оформлению
ГОСТ 34.602-2020	Техническое задание на создание автоматизированной системы

Конкретный состав и шаблоны документов определяются регламентом курса и указаниями преподавателя.



Домашнее задание =)

1 Найти друзей



Домашнее задание =)

- 1 Создать команду из двух или трёх человек;
- 2 Придумать идею для проекта;
- 3 Написать в ЛС преподавателю, указав состав команды (тг + фио) и выбранную тему проекта;
- 4 Набросать черновик ТЗ с предметной областью, целью, функциональными и нефункциональными требованиями;
- 5 Принести его на следующую лабу для обсуждения и согласования.

